

deploy.sh

```
$ ./deploy.sh --env prod
> backup complete
> status: success ✓
$ for s in web1 web2 db1
> checking web1 ... ok
> checking web2 ... ok
> checking db1 ... ok
$ _
```

```
#!/bin/bash
name="Priya"
for i in 1 2 3
do
    echo $i
done
.sh
```

Shell Scripting for DevOps

A Complete Beginner-to-Advanced Guide

Based on the open-source repository by LondheShubham153 (TrainWithShubham)
github.com/LondheShubham153/Shell-Scripting-For-DevOps

PDF CRAFTED BY



Sharad Verma

@sharadverma638

19 LESSONS

REAL DEVOPS USE-CASES

CLOUD BONUS

CHEAT SHEET

19

lessons

25+

illustrations

1

grand script



Want to learn DevOps?



github.com/sharadverma638/sv-opslibrary

Contents

12 sections, 19 lessons. Follow in order, or jump to what you need.

01	How to Use This Guide + Roadmap Your learning path and how each lesson works	2
02	Phase 1: Output, Variables & Input Lessons 1-4	3
03	Phase 2: File Operations & Operators Lessons 5-6	7
04	Phase 3: Conditionals Lessons 7-9	9
05	Phase 4: Loops Lessons 10-12	12
06	Phase 5: Functions Lessons 13-15	15
07	Phase 6: Real DevOps Tasks Lessons 16-18	18
08	Phase 7: Capstone Project Backup script + cron (Lesson 19)	21
09	Bonus: Cloud Automation Scripts Docker + AWS EC2	22
10	Additional Topics Apart from the repo: must-know skills	23
11	The Grand Script All lessons combined + line-by-line explanation	28
12	Quick Revision Cheat Sheet Everything on one page, fast	33

Lessons at a glance

1 Hello World	2 Variables	3 Arguments	4 read input	5 && and
6 Install pkgs	7 if-else	8 File exists	9 if-elif-else	10 for list
11 for counter	12 while	13 Functions	14 Func args	15 source
16 Create user	17 awk info	18 Errors	19 Backup+cron	

01

How to Use This Guide + Roadmap

Your learning path and how each lesson works

This guide follows a step-by-step path: from your very first line of code to a real automated backup system with cloud deployment. **Follow it in order; each lesson builds on the one before it.**

Learning roadmap

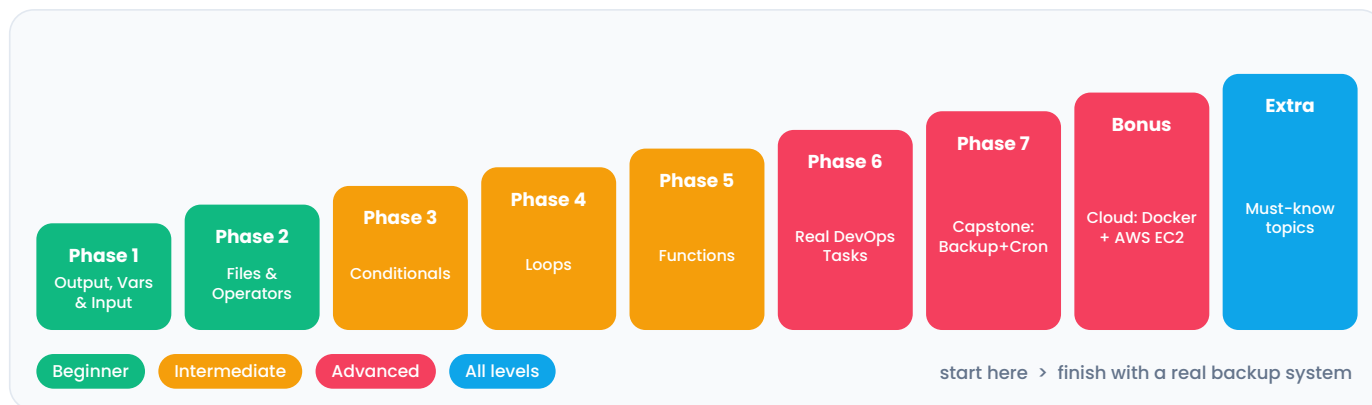


Fig 1. Seven phases, a cloud bonus and extra must-know topics.

Stage	Topic	Covers	Level
Phase 1	Output, Variables & Input	Lessons 1-4	Beginner
Phase 2	File Operations & Operators	Lessons 5-6	Beginner
Phase 3	Conditionals	Lessons 7-9	Intermediate
Phase 4	Loops	Lessons 10-12	Intermediate
Phase 5	Functions	Lessons 13-15	Intermediate
Phase 6	Real DevOps Tasks	Lessons 16-18	Advanced
Phase 7	Capstone: Backup + Cron	Lesson 19	Advanced
Bonus	Cloud Automation (Docker + AWS EC2)	day03/ scripts	Advanced
Extra	Additional Must-Know Topics	Not in repo, added here	All Levels

How each lesson is structured



Fig 2. Every lesson follows the same seven steps.

- **What It's About:** a plain-language introduction to the concept
- **Syntax:** the bare-bones pattern to remember
- **Use In Real DevOps Life:** where this is actually used on the job
- **Task:** a small exercise to practice the concept yourself
- **Key Learning:** the exact things you will pick up
- **Easy Example:** a working script you can run as-is
- **Key Takeaways:** the most important points to remember

Original repository: <https://github.com/LondheShubham153/Shell-Scripting-For-DevOps>

02

Phase 1: Output, Variables & Input

Lessons 1-4



01

LESSON 1

Your First Script – Hello World

01_hello.sh

```
#!/bin/bash
# This is a comment
echo "text to print"
```

- **Shebang**
tells Linux which interpreter to use. Must be line 1.
- **Comment**
ignored by bash. Notes for your teammates.
- **echo**
prints text to the screen.

WHAT IS THIS LESSON ABOUT?

This is the very first script every learner writes. It introduces the shebang line, the echo command, and comments – the three building blocks every shell script starts with.

KEY LEARNING

- Shebang line `#!/bin/bash`
- `echo` command to print text
- Comments using `#`

SYNTAX / PATTERN

```
#!/bin/bash
# This is a comment
echo "text to print"
```

EASY EXAMPLE

```
#!/bin/bash
# My first shell script
echo "Namaste! Aaj hum shell scripting seekhenge."
echo "Mera pehla balance hai Rs 5000"
```

USE IN REAL DEVOPS LIFE

Every automation script (deployment, backup, monitoring) begins with a shebang line. Without it, Linux would not know which interpreter to use to run your file.

KEY TAKEAWAYS

- Shebang must be the very first line of the file
- `echo` is the simplest way to give output/feedback from a script
- Comments make scripts readable for your teammates

YOUR TASK

Create `wallet.sh` that prints your name and how much money (Rs) you have.

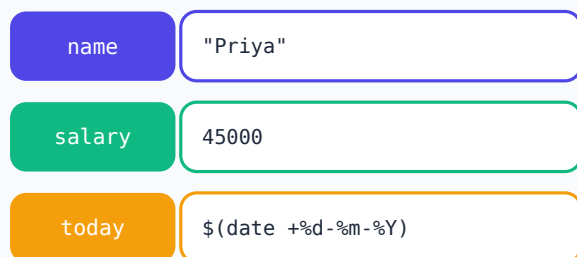
02

LESSON 2

Variables & Command Substitution

02_variables.sh

STORE



READ with \$

echo "Namaste \$name!"

echo "Salary Rs \$salary"

echo "Date: \$today"

`$( )` runs the command first, then stores its output

WHAT IS THIS LESSON ABOUT?

Variables let a script store and reuse data – names, numbers, dates, file paths. Command substitution lets you capture the output of a command into a variable.

KEY LEARNING

- Assigning a variable: `name="value"`
- Reading a variable: `$name`
- Command substitution: `$(command)`

SYNTAX / PATTERN

```
name="value"
echo "$name"
today=$(date)
```

EASY EXAMPLE

```
#!/bin/bash
name="Priya"
salary=45000
today=$(date +%d-%m-%Y)
echo "Namaste $name!"
echo "Aapki salary hai Rs $salary"
echo "Aaj ki date hai: $today"
```

USE IN REAL DEVOPS LIFE

Scripts constantly store server names, file paths, timestamps and counts in variables – for example naming a backup file using today's date.

KEY TAKEAWAYS

- Never put spaces around the `=` sign
- Always wrap variables in quotes: `"$name"`
- `$(command)` runs a command first, then stores its output

YOUR TASK

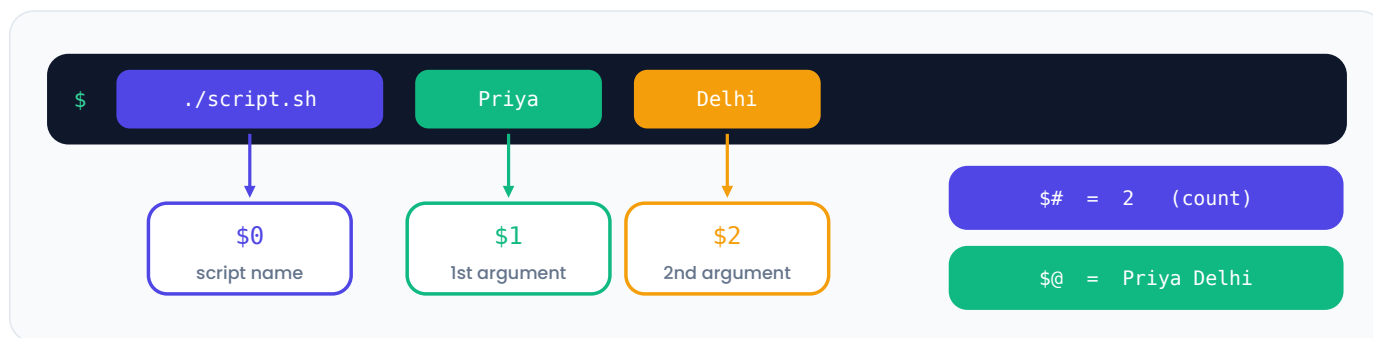
Write `shop.sh` with `item="Kurta"` and `price=1200`. Print the price and also store today's date using `$(date)`.

03

LESSON 3

Script Arguments

03_arguments.sh



WHAT IS THIS LESSON ABOUT?

Arguments let you pass values into a script directly from the command line, at the moment you run it – without the script having to ask for input.

KEY LEARNING

- \$0 = script name
- \$1, \$2... = 1st, 2nd argument
- \$# = count of arguments
- \$@ = all arguments

SYNTAX / PATTERN

```
./script.sh arg1 arg2  
# inside script:  
echo $0 $1 $2 $# $@
```

EASY EXAMPLE

```
#!/bin/bash  
echo "Script name: $0"  
echo "Customer name: $1"  
echo "City: $2"  
echo "Total arguments given: $#"  
echo "All arguments: $@"
```

USE IN REAL DEVOPS LIFE

Deployment scripts commonly take the environment name or version number as an argument, e.g. `./deploy.sh production v2.3`

KEY TAKEAWAYS

- Arguments are given at run time, not while the script runs
- \$# helps you validate that the user gave enough input
- Missing arguments are simply empty, not an error, unless you check for them

YOUR TASK

Write `recharge.sh` that takes a mobile number as \$1 and amount as \$2, and prints a confirmation message.

04

LESSON 4

User Input with read

04_user_input.sh

ARGUMENTS (\$1, \$2)

```
$ ./deploy.sh production v2.3
```

values are given BEFORE the script runs
best for automation

read -p (asks the user)

```
Enter your name: Priya_
```

script asks WHILE it is running
best for interactive scripts

WHAT IS THIS LESSON ABOUT?

Sometimes you want to ask the user a question WHILE the script is running, instead of requiring arguments upfront. The read command does exactly this.

KEY LEARNING

- `read -p "prompt" variable`
- Difference between `read` and `$1/$2`

SYNTAX / PATTERN

```
read -p "Enter your name: " name
```

EASY EXAMPLE

```
#!/bin/bash
read -p "Enter your name: " name
read -p "Enter your monthly salary (Rs): " salary
echo "Namaste $name!"
echo "Aapki salary hai Rs $salary"
```

USE IN REAL DEVOPS LIFE

Interactive setup scripts (like first-time server configuration wizards) use read to collect information from an admin step by step.

KEY TAKEAWAYS

- `-p` shows a prompt message to the user
- The value typed by the user gets stored in the variable name you specify
- Good for interactive scripts; arguments are better for automation

YOUR TASK

Write `atm.sh` that asks for name and withdrawal amount using read, then prints a confirmation.

03

Phase 2: File Operations & Operators

Lessons 5-6

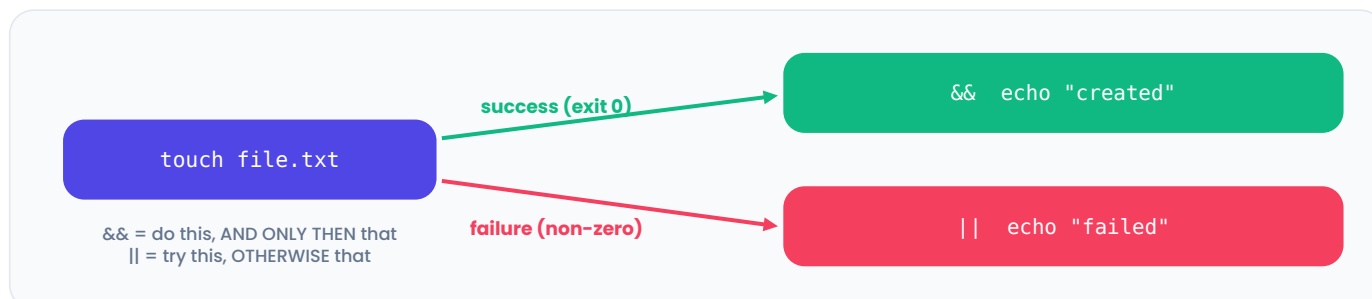


05

LESSON 5

File Creation & Operators

05_create_file.sh



WHAT IS THIS LESSON ABOUT?

This lesson introduces creating files with `touch`, and controlling the flow of a script based on whether a command succeeded or failed, using `&&` and `||`.

KEY LEARNING

- `touch filename` to create an empty file
- `&&` runs the next command only on success
- `||` runs the next command only on failure

SYNTAX / PATTERN

```
touch file.txt && echo "created" || echo "failed"
```

EASY EXAMPLE

```
#!/bin/bash
filename="invoice.txt"
touch "$filename" && echo "File created!" || echo "Failed to create file"
```

USE IN REAL DEVOPS LIFE

Deployment scripts use `&&` to chain steps that must all succeed in order (build && test && deploy), and `||` to trigger alerts when a step fails.

KEY TAKEAWAYS

- `touch` does not erase an existing file's contents
- `&&` = do this, AND ONLY THEN do that
- `||` = try this, OTHERWISE do that

YOUR TASK

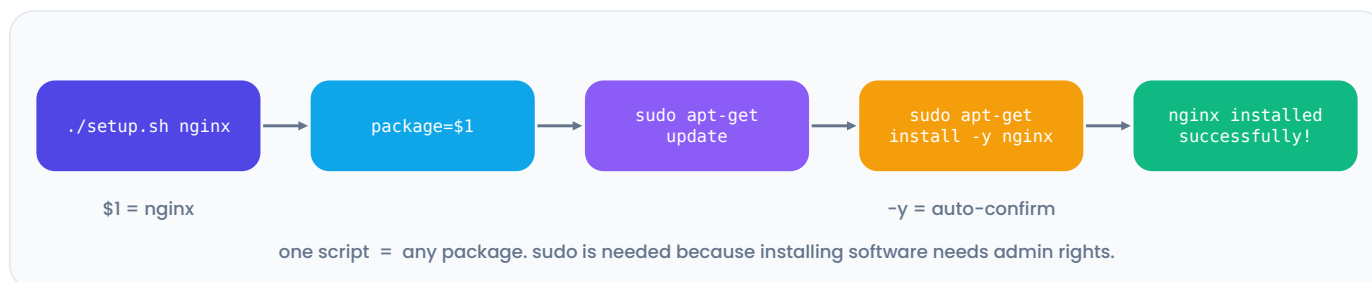
Write `paytm.sh` that creates `transaction.txt` with `touch`, printing success or failure using `&&` and `||`.

06

LESSON 6

Installing Packages

06_install_package.sh



WHAT IS THIS LESSON ABOUT?

A reusable script that installs any software package you give it as an argument, using apt-get. This turns a manual, repeated task into a one-line automation.

KEY LEARNING

- `sudo apt-get update / install -y`
- Using `$1` to make the script generic (reusable for any package)

SYNTAX / PATTERN

```
package=$1
sudo apt-get install -y "$package"
```

EASY EXAMPLE

```
#!/bin/bash
package=$1
echo "Installing $package..."
sudo apt-get update
sudo apt-get install -y "$package"
echo "$package installed successfully!"
```

USE IN REAL DEVOPS LIFE

Server provisioning scripts install a whole stack of tools (git, docker, nginx, awscli) automatically when a new machine is set up.

KEY TAKEAWAYS

- `sudo` is required because installing software needs admin rights
- `-y` auto-confirms the installation so the script doesn't pause
- This script pattern works on any Debian/Ubuntu based system

YOUR TASK

Write `setup.sh` that installs a package given as `$1`, with a 'please wait' message and a short delay before confirming.

04

Phase 3: Conditionals

Lessons 7-9

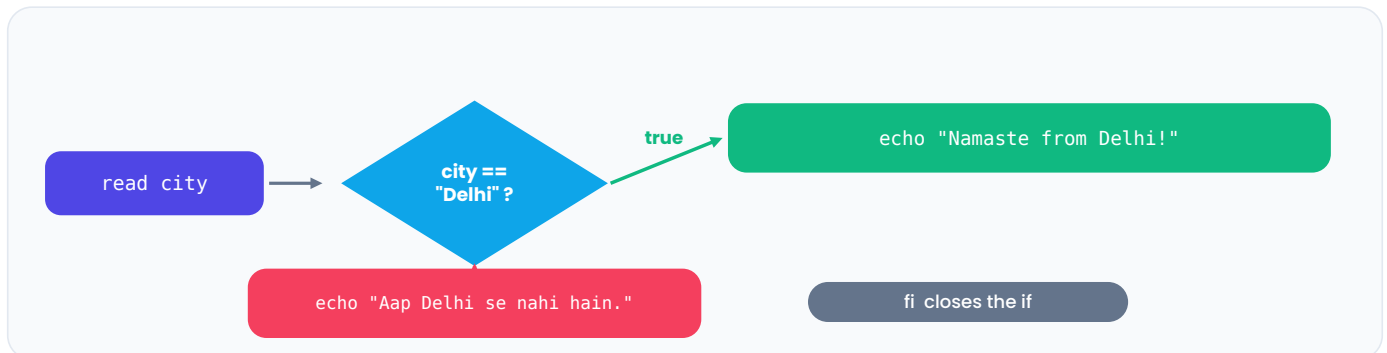


07

LESSON 7

If-Else Conditionals

07_if_else.sh



WHAT IS THIS LESSON ABOUT?

Conditionals let a script make decisions – running different code depending on whether something is true or false.

KEY LEARNING

- `if [[ condition ]] then ... else ... fi`
- String comparison: `==` and `!=`
- Numeric comparison: `-eq -ne -ge -le -gt -lt`

SYNTAX / PATTERN

```

if [[ "$var" == "value" ]]
then
    # true block
else
    # false block
fi
  
```

EASY EXAMPLE

```

#!/bin/bash
read -p "Enter your city: " city
if [[ "$city" == "Delhi" ]]
then
    echo "Namaste from Delhi!"
else
    echo "Aap Delhi se nahi hain."
fi
  
```

USE IN REAL DEVOPS LIFE

Deployment scripts check conditions constantly – 'if the build succeeded', 'if disk space is above 80%', 'if the environment is production'.

KEY TAKEAWAYS

- Use `==` / `!=` only for text (strings)
- Use `-eq -ne -gt -lt -ge -le` only for numbers
- `fi` always closes an if block (if spelled backwards)

YOUR TASK

Write `discount.sh`: if purchase amount is 1000 or more, print a 10% discount message, else no discount.

08

LESSON 8

Checking if a File Exists

08_if_file_exists.sh

FILE TEST OPERATORS

-f

regular file

-d

directory

-e

exists (any)

-s

not empty

-r

readable

-w

writable

-x

executable

EXIT CODES

0success
exit 0**1**failure
exit 1

WHAT IS THIS LESSON ABOUT?

Before working with a file, a good script checks whether it actually exists - using the `-f` file test. This lesson also introduces stopping a script safely with `exit`.

KEY LEARNING

- File test operators: `-f -d -e -s -r -w -x`
- Checking argument count with `$#`
- `exit 0` (success) vs `exit 1` (failure)

SYNTAX / PATTERN

```
if [[ -f "$filename" ]]
then
    echo "exists"
fi
```

EASY EXAMPLE

```
#!/bin/bash
if [[ $# -eq 0 ]]
then
    echo "Error: Please provide a filename."
    exit 1
fi
filename=$1
if [[ -f "$filename" ]]
then
    echo "$filename exists."
else
    echo "$filename does not exist."
fi
```

USE IN REAL DEVOPS LIFE

Backup and deployment scripts always check that a required config file or folder exists before proceeding, to avoid crashing halfway through.

KEY TAKEAWAYS

- `-f` checks a regular file, `-d` checks a directory
- `$#` tells you how many arguments were given, useful for validation
- `exit 1` signals failure to whoever runs or monitors the script

YOUR TASK

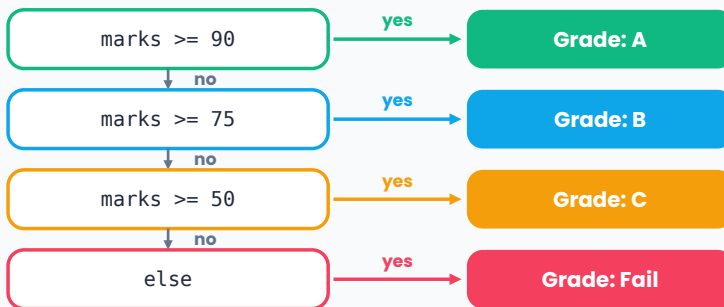
Write `check_backup.sh`: validate `$#` is not 0, then check if the given filename exists using `-f`.

09

LESSON 9

Multi-branch: if-elif-else

09_if_elif_else.sh

**Checked top to bottom.**

First true match wins,
the rest are skipped.

WHAT IS THIS LESSON ABOUT?

When there are more than two possible outcomes, elif lets you check multiple conditions in order, stopping at the first true match.

KEY LEARNING

- `elif` keyword for extra conditions
- Order of conditions matters (top to bottom)

SYNTAX / PATTERN

```
if [[ cond1 ]]
then
    ...
elif [[ cond2 ]]
then
    ...
else
    ...
fi
```

EASY EXAMPLE

```
#!/bin/bash
read -p "Enter your marks: " marks
if [[ $marks -ge 90 ]]
then
    echo "Grade: A"
elif [[ $marks -ge 75 ]]
then
    echo "Grade: B"
elif [[ $marks -ge 50 ]]
then
    echo "Grade: C"
else
    echo "Grade: Fail"
fi
```

USE IN REAL DEVOPS LIFE

Health-check scripts use elif to classify server status into multiple levels, e.g. Healthy / Warning / Critical, based on CPU or memory usage.

KEY TAKEAWAYS

- Bash checks conditions top to bottom and stops at the first match
- Always write the strictest/highest condition first
- `elif` avoids deeply nested if-inside-if code

YOUR TASK

Write `movie_ticket.sh`: print a ticket price based on age, with 4 different price bands.

05

Phase 4: Loops

Lessons 10-12



10

LESSON 10

for Loop (over a list)

10_for_loop_list.sh

for user in

ravi

priya

amit

sneha

```
mkdir "$user"
echo "Folder created for $user"
done
```

runs 4 times

once per item

WHAT IS THIS LESSON ABOUT?

Loops repeat a block of code automatically instead of writing it manually many times. This lesson covers looping over a fixed list of items or files.

KEY LEARNING

- for item in list do ... done
- Glob patterns like *.txt to loop over files

SYNTAX / PATTERN

```
for item in item1 item2 item3
do
    echo "$item"
done
```

EASY EXAMPLE

```
#!/bin/bash
for user in ravi priya amit sneha
do
    mkdir "$user"
    echo "Folder created for $user"
done
```

USE IN REAL DEVOPS LIFE

Used to loop through a list of servers, users, or files – e.g. restarting a service on 5 different servers with one script.

KEY TAKEAWAYS

- do and done mark the start and end of the loop body
- *.txt style patterns loop over matching files automatically
- Great when you already know the exact list of items

YOUR TASK

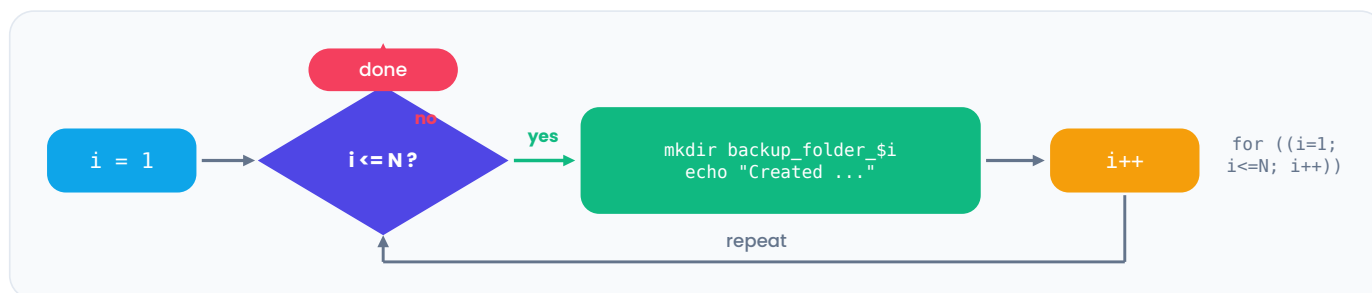
Write `salary_slip.sh` that loops through 4 employee names and prints a slip-generated message for each.

11

LESSON 11

C-style for Loop (counters)

11_for_loop_counter.sh



WHAT IS THIS LESSON ABOUT?

When you need to repeat something a specific number of times (not a fixed list), the C-style for loop with a counter is the right tool.

KEY LEARNING

- `for (( i=1; i<=N; i++ )) do ... done`
- Start value, condition, increment

SYNTAX / PATTERN

```
for (( i=1; i<=5; i++ ))
do
    echo "Count: $i"
done
```

EASY EXAMPLE

```
#!/bin/bash
count=$1
for (( i=1; i<=count; i++ ))
do
    mkdir "backup_folder_$i"
    echo "Created backup_folder_$i"
done
```

USE IN REAL DEVOPS LIFE

Used to create N numbered resources – N backup folders, N test users, or to retry an action a fixed number of times.

KEY TAKEAWAYS

- Same for loop syntax exists in C, Java and JavaScript too
- `i++` increases the counter by 1 after every loop
- Best choice when you need to count or repeat N times

YOUR TASK

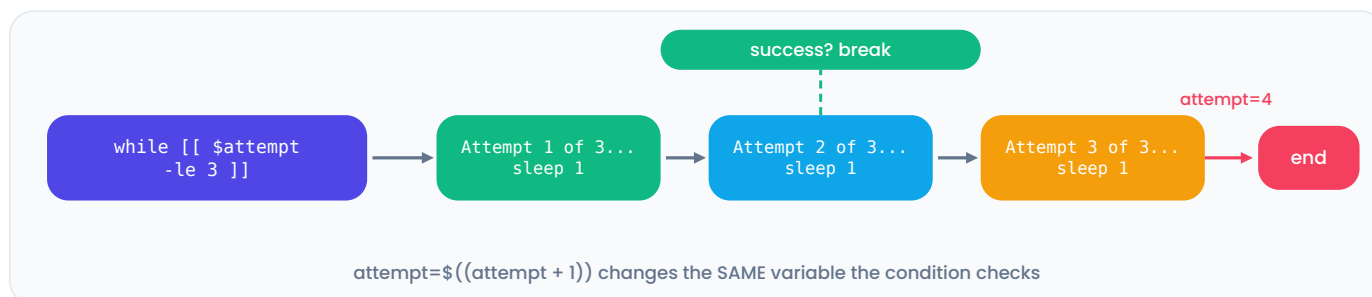
Write `emi_reminder.sh` that prints an EMI reminder message for each month, up to the number given as `$1`.

12

LESSON 12

while Loop

12_while_loop.sh



WHAT IS THIS LESSON ABOUT?

A while loop keeps running as long as a condition stays true. Useful when you don't know in advance exactly how many times you'll loop – like retrying until success.

KEY LEARNING

- while [[condition]] do ... done
- Arithmetic: `$(expression)`
- break and continue
- Infinite loops: while true

SYNTAX / PATTERN

```
while [[ $i -le 5 ]]
do
    echo "$i"
    i=$((i + 1))
done
```

EASY EXAMPLE

```
#!/bin/bash
attempt=1
max_attempts=3
while [[ $attempt -le $max_attempts ]]
do
    echo "Attempt $attempt of $max_attempts..."
    attempt=$((attempt + 1))
    sleep 1
done
```

USE IN REAL DEVOPS LIFE

Retry logic (retry a failed deployment 3 times), and polling loops (wait for a server to be ready) both use while loops – this is the exact pattern used in the advanced AWS EC2 script.

KEY TAKEAWAYS

- The variable that changes inside the loop must be the same one checked in the condition
- while true creates an infinite loop – use break to exit or Ctrl+C to stop manually
- `$(...)` is required for any arithmetic in Bash

YOUR TASK

Write `countdown.sh` that counts down from 10 to 1 using a while loop, then prints a final message.

06

Phase 5: Functions

Lessons 13-15



13

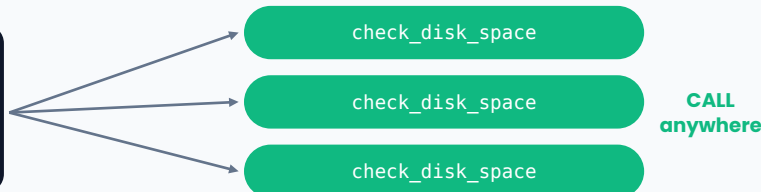
LESSON 13

Functions

13_functions.sh

DEFINE ONCE

```
check_disk_space() {  
    echo "Checking..."  
    df -h  
}
```



WHAT IS THIS LESSON ABOUT?

Functions let you write a block of code once and reuse it anywhere in your script, instead of copy-pasting the same lines repeatedly.

KEY LEARNING

- Defining: `function_name() { ... }`
- Calling: just `function_name`
- Functions must be defined before they are called

SYNTAX / PATTERN

```
greet() {  
    echo "Hello!"  
}  
greet
```

EASY EXAMPLE

```
#!/bin/bash  
check_disk_space() {  
    echo "Checking disk space..."  
    df -h  
}  
check_memory() {  
    echo "Checking memory..."  
    free -h  
}  
echo "--- System Health Check ---"  
check_disk_space  
check_memory
```

USE IN REAL DEVOPS LIFE

Almost every real automation script organizes its logic into functions like `check_status()`, `send_alert()`, `create_backup()` for clarity and reuse.

KEY TAKEAWAYS

- Functions make long scripts easier to read and maintain
- A function must be written above the line where you call it
- You call a function just by typing its name, no `()` needed at call time

YOUR TASK

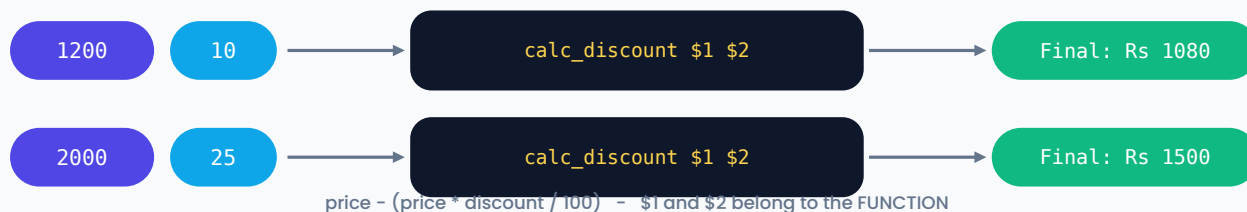
Write `bank.sh` with a function `show_balance`, and call it twice in the script.

14

LESSON 14

Functions with Arguments

14_function_with_args.sh



WHAT IS THIS LESSON ABOUT?

Just like a script can take arguments (`$1`, `$2`), a function can too - letting the same function work with different values each time it's called.

KEY LEARNING

- Inside a function, `$1` refers to the function's own argument, not the script's
- Passing multiple values: `func_name arg1 arg2`

SYNTAX / PATTERN

```
greet() {  
  echo "Namaste, $1!"  
}  
greet "Rohan"
```

EASY EXAMPLE

```
#!/bin/bash  
calc_discount() {  
  price=$1  
  discount=$2  
  final=$(( price - (price * discount / 100) ))  
  echo "Final price: Rs $final"  
}  
calc_discount 1200 10  
calc_discount 2000 25
```

USE IN REAL DEVOPS LIFE

A reusable function like `create_user_folder("$username")` or `deploy_service("$service_name")` can be called for many different values in a loop.

KEY TAKEAWAYS

- `$1` means something different inside vs outside a function
- This is what makes a function truly reusable, not just repeatable
- Combine this with a for loop to apply the same function to many items

YOUR TASK

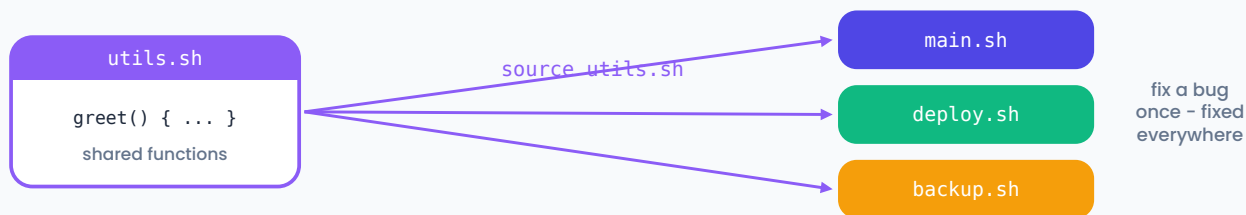
Write a function `calc_discount(price, discount)` using arguments (not read), and call it for a few items.

15

LESSON 15

source – Reusing Code Across Scripts

15_source_and_call.sh



WHAT IS THIS LESSON ABOUT?

Instead of copy-pasting the same functions into every script, you can write them once in a shared file and load ('source') them into any script that needs them.

KEY LEARNING

- `source filename.sh` Or `. filename.sh`
- Sharing functions/variables across multiple scripts

SYNTAX / PATTERN

```
source utils.sh
greet "Rohan"
```

EASY EXAMPLE

```
# utils.sh
greet() {
  echo "Namaste, $1!"
}

# main.sh
source utils.sh
greet "Rohan"
```

USE IN REAL DEVOPS LIFE

Real projects keep a `common_functions.sh` with logging, alerting and checking functions, and every other script just sources it at the top.

KEY TAKEAWAYS

- `source` runs the file's code as if it were pasted directly into your script
- A sourced file does not need `chmod +x` or even a shebang
- Fixing a bug in the shared file fixes it everywhere it's used

YOUR TASK

Create `math_utils.sh` with an `add_numbers` function, then source it from `test.sh` and call it.

07

Phase 6: Real DevOps Tasks

Lessons 16-18

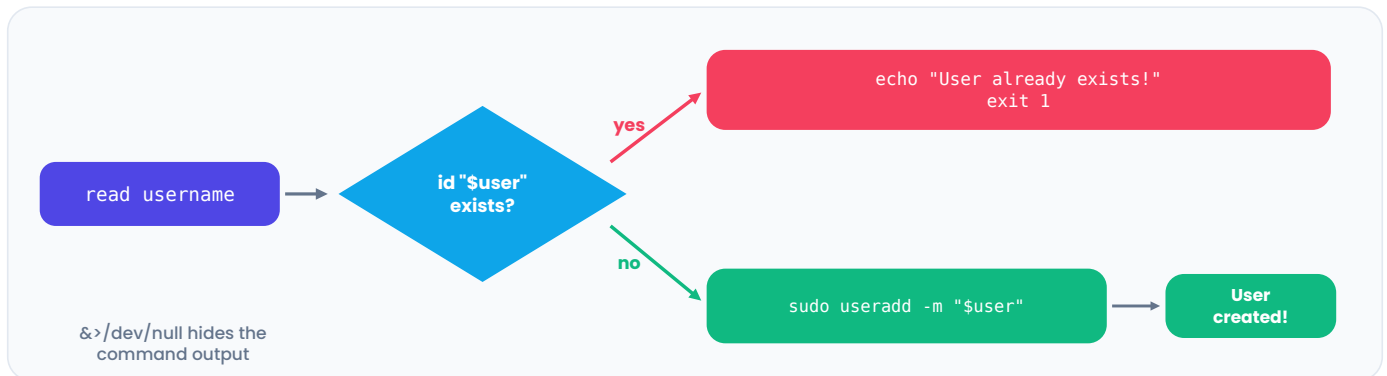


16

LESSON 16

Creating a Linux User

16_create_user.sh



WHAT IS THIS LESSON ABOUT?

A complete real workflow: take input, check if a user already exists, and only then create it. Combines read, if, exit and a new command called id.

KEY LEARNING

- `id username` to check if a user exists
- `useradd -m` to create a user with a home folder
- `&>/dev/null` to hide command output

SYNTAX / PATTERN

```
if id "$username" &>/dev/null
then
    exit 1
else
    sudo useradd -m "$username"
fi
```

EASY EXAMPLE

```
#!/bin/bash
read -p "Enter username to create: " username
if id "$username" &>/dev/null
then
    echo "User already exists!"
    exit 1
else
    sudo useradd -m "$username"
    echo "User created successfully!"
fi
```

USE IN REAL DEVOPS LIFE

Onboarding automation - creating a Linux user account for every new team member on a shared server, safely and without duplicates.

KEY TAKEAWAYS

- Always check before creating, to avoid errors or duplicates
- `/dev/null` is like a trash bin for output you don't need to see
- `useradd` needs `sudo` because it changes system-level settings

YOUR TASK

Extend the script to also delete a user if a second argument `--delete` is given.

17

LESSON 17

System Info with awk

17_system_details.sh

free -h

awk 'NR==2 {print \$3}'

4.2Gi

only the number
you need

	\$1	\$2	\$3	\$4
NR=1		total	used	free
NR=2	Mem:	15Gi	4.2Gi	8.1Gi
NR=3	Swap:	2.0Gi	0B	2.0Gi

NR==2 = row 2
\$3 = column 3
awk = Excel column
selector for text

WHAT IS THIS LESSON ABOUT?

Pipes connect commands together, and awk extracts specific columns from command output – turning a big block of text into just the one number you need.

KEY LEARNING

- Pipes: `command1 | command2`
- `awk '{print $N}'` to print column N
- `NR` for row number, `-F` to change separator

SYNTAX / PATTERN

```
df -h | awk 'NR==2 {print $5}'
```

EASY EXAMPLE

```
#!/bin/bash
echo -n "Memory used: "
free -h | awk 'NR==2 {print $3 " / " $2}'
echo -n "Disk used: "
df -h / | awk 'NR==2 {print $5}'
```

USE IN REAL DEVOPS LIFE

Monitoring scripts constantly extract just the CPU load, memory used, or disk usage percentage to log it or trigger an alert if it crosses a threshold.

KEY TAKEAWAYS

- A pipe sends the output of one command into the next command
- awk works like Excel's column selector for terminal text
- `$1` `$2` `$3` in awk mean column 1, 2, 3 – not the same as script arguments

YOUR TASK

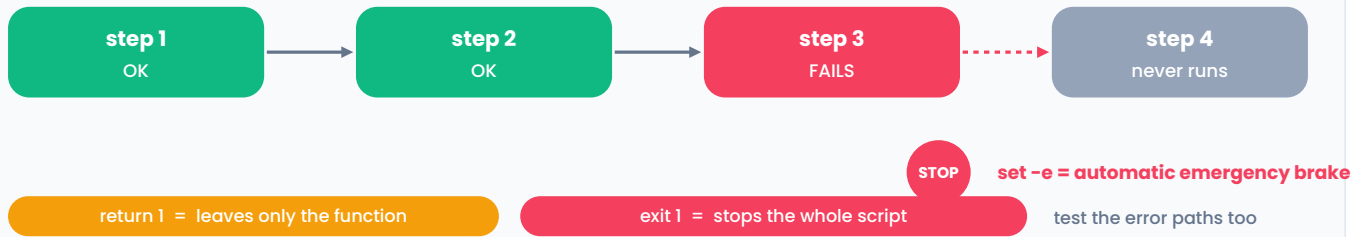
Write `disk_alert.sh` that stores the disk usage percentage in a variable using `df` and `awk`.

18

LESSON 18

Error Handling

18_error_handling.sh



WHAT IS THIS LESSON ABOUT?

Good automation scripts stop safely the moment something goes wrong, instead of continuing and causing more damage. This lesson covers `set -e` and manual failure checks.

KEY LEARNING

- `set -e` stops the script automatically on any failed command
- `if ! command; then ...` – checking for failure manually
- `return` vs `exit` inside functions

SYNTAX / PATTERN

```
set -e
if ! command
then
    exit 1
fi
```

EASY EXAMPLE

```
#!/bin/bash
set -e
check_folder() {
    if [[ ! -d "$1" ]]
    then
        echo "Folder $1 does not exist!"
        return 1
    fi
}
if ! check_folder "/home/user/documents"
then
    exit 1
fi
echo "Continuing with backup..."
```

USE IN REAL DEVOPS LIFE

Production deployment and backup scripts always use `set -e` so that a single failed step (like a missing folder) stops the whole pipeline instead of silently continuing.

KEY TAKEAWAYS

- `set -e` is like an automatic emergency brake for your script
- `return 1` stops only a function; `exit 1` stops the whole script
- Always test error paths, not just the happy path

YOUR TASK

Add `set -e` to your `create_user.sh` and confirm it stops immediately if a non-existent command runs.

08

Phase 7: Capstone Project

Backup script + cron (Lesson 19)



19

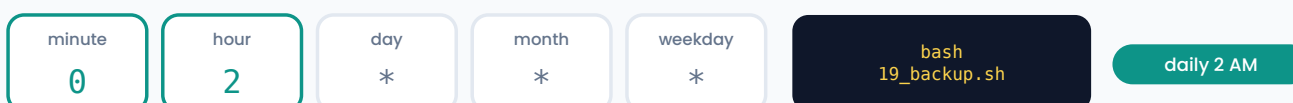
LESSON 19

Capstone: Backup Script + Cron

19_backup.sh



CRON = runs it automatically (crontab -e)



WHAT IS THIS LESSON ABOUT?

The final project combines everything – variables, functions, conditionals, error handling and command substitution – into one real backup automation script, then schedules it to run automatically using cron.

KEY LEARNING

- `tar -czf` to create a compressed backup archive
- Timestamped filenames using `$(date +%Y%m%d_%H%M%S)`
- Scheduling automatic runs with `crontab -e`

SYNTAX / PATTERN

```
tar -czf backup_$(date +%Y%m%d_%H%M%S).tar.gz /path/to/folder
```

EASY EXAMPLE

```
#!/bin/bash
set -e
BACKUP_DIR="$HOME/shell-backups"
create_backup() {
  mkdir -p "$BACKUP_DIR"
  timestamp=$(date +%Y%m%d_%H%M%S)
  tar -czf "$BACKUP_DIR/backup_${timestamp}.tar.gz" ./practice-scripts
  echo "Backup created: backup_${timestamp}.tar.gz"
}
create_backup
```

USE IN REAL DEVOPS LIFE

This exact pattern (backup + retention + cron) is used in real production servers to protect databases, configs and code from data loss, running silently every night.

KEY TAKEAWAYS

- This script is a real combination of every lesson from 1 to 18
- Cron format is: `minute hour day month weekday command`
- Always test a cron job manually before scheduling it

YOUR TASK

Schedule your backup script to run daily at 2 AM using `crontab -e`, and verify with `crontab -l`.

09

Bonus: Cloud Automation Scripts

Docker + AWS EC2

These two advanced scripts live in the repository's `day03/` folder. They need an AWS account and `aws-cli` configured, and combine everything from earlier lessons with real cloud tools.

Docker deployment script

File: `day03/deploy_django_app.sh`

A multi-step script that pulls code from a Git repository and deploys it as a Docker container – combining **git**, **docker build**, and **docker run** commands into one automated flow.

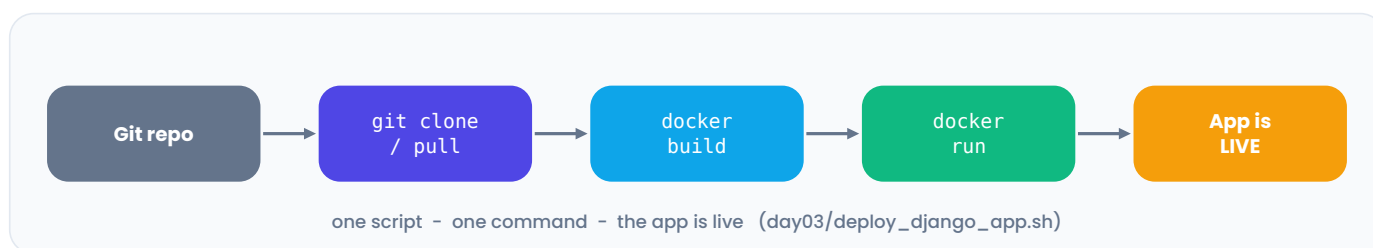


Fig 3. From Git repository to a live container.

REAL USE

This is exactly how many real teams deploy web applications – one script, one command, and the app is live.

AWS EC2 provisioning script

File: `day03/create_ec2.sh`

Provisions a brand-new AWS EC2 server end-to-end using the AWS CLI, then uses a polling **while** loop (same pattern as Lesson 12) to wait until the server is ready before continuing.

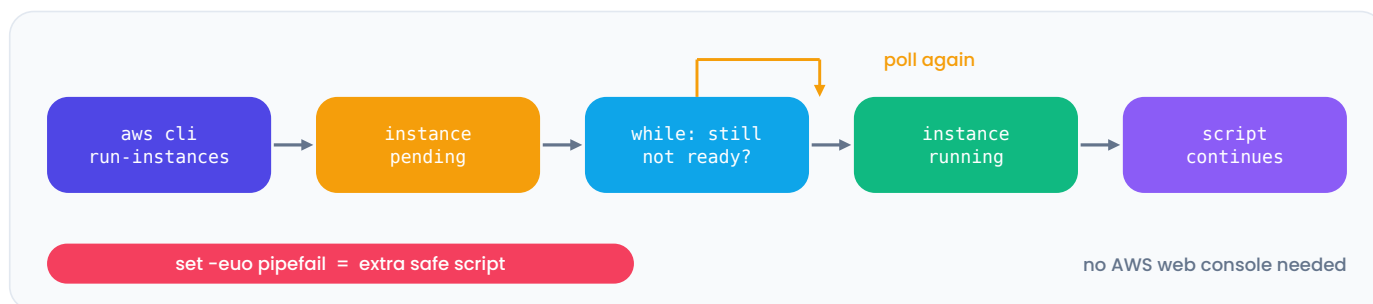


Fig 4. Create the server, then poll until it is ready.

REAL USE

Infrastructure automation – spinning up new cloud servers without ever touching the AWS web console. Requires an AWS account and `aws-cli` configured with `set -euo pipefail` for safety.

10

Additional Topics

Apart from the repo: must-know skills



These topics are not part of the original repository, but every DevOps engineer ends up using them regularly. Learning these on top of the 19 lessons will make you job-ready.

1. case Statement (cleaner multi-choice menus)

An alternative to long if-elif-else chains when checking one variable against many fixed values – commonly used for menu-driven scripts.



```
#!/bin/bash
read -p "Choose (start/stop/status): " action
case $action in
  start)
    echo "Starting service..." ;;
  stop)
    echo "Stopping service..." ;;
  status)
    echo "Checking status..." ;;
  *)
    echo "Invalid option" ;;
esac
```

REAL USE

Service-control scripts (start/stop/restart/status) almost always use case instead of if-elif, since it is shorter and easier to read.

2. Arrays

Arrays let you store multiple values in a single variable – useful for lists of servers, filenames, or usernames.

```
servers=("web1" "web2" "db1")
```

web1

web2

db1

[0]

[1]

[2]

`${servers[0]}`

= web1 (first)

`${servers[@]}`

= all items

`${#servers[@]}`

= 3 (how many)

```
#!/bin/bash
servers=("web1" "web2" "db1")
echo "First server: ${servers[0]}"
echo "All servers: ${servers[@]}"
echo "Total servers: ${#servers[@]}"
for s in "${servers[@]}"
do
  echo "Checking $s..."
done
```

REAL USE

Storing a list of servers to loop through and run health checks or deployments on each one.

3. grep – Searching Inside Files

grep searches text for lines matching a pattern – one of the most-used commands in all of DevOps, especially for reading log files.



```
grep "ERROR" app.log
grep -i "error" app.log    # case-insensitive
grep -c "ERROR" app.log    # count matches
grep -r "TODO" ./project   # search recursively in a folder
```

REAL USE

Searching server logs for errors, or finding which files in a project contain a certain configuration value.

4. sed – Find and Replace in Files

sed (stream editor) is used to find and replace text inside files directly from the command line, without opening an editor.



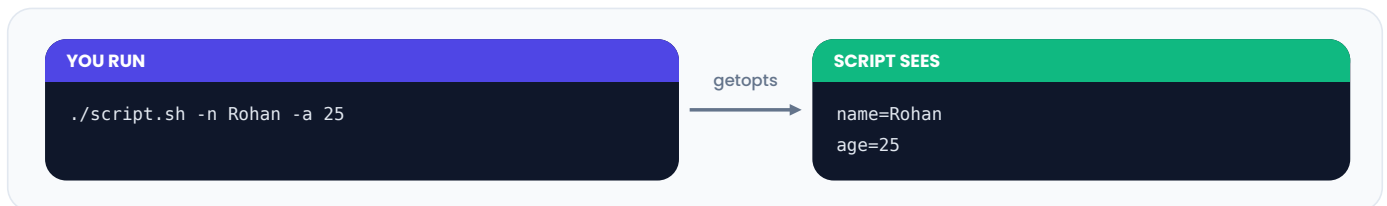
```
sed 's/old-text/new-text/' file.txt    # preview change
sed -i 's/old-text/new-text/' file.txt  # apply change in file
sed -i 's/DEBUG=false/DEBUG=true/' config.env
```

REAL USE

Updating a config file's value (like a version number or environment flag) automatically during a deployment, without manual editing.

5. getopt - Proper Command-Line Flags

Real command-line tools accept flags like `-n` or `--name`, not just plain arguments. `getopts` is Bash's built-in way to handle this properly.



```
#!/bin/bash
while getopts "n:a:" opt
do
    case $opt in
        n) name=$OPTARG ;;
        a) age=$OPTARG ;;
    esac
done
echo "Name: $name, Age: $age"
# Run as: ./script.sh -n Rohan -a 25
```

REAL USE

Professional CLI tools (like the scripts DevOps engineers write for their teams) almost always use flags instead of plain positional arguments, since it is less error-prone.

6. trap - Handling Signals & Cleanup

`trap` lets a script run cleanup code automatically when it is interrupted (like `Ctrl+C`) or exits - useful for removing temporary files even if something goes wrong.



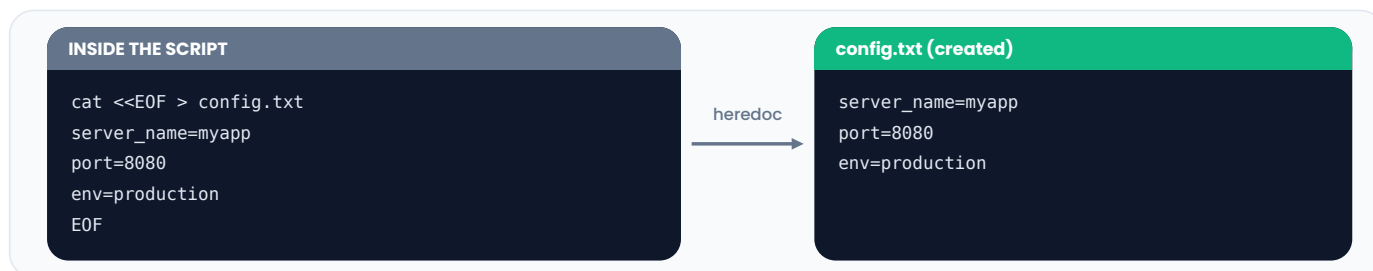
```
#!/bin/bash
cleanup() {
    echo "Cleaning up temporary files..."
    rm -f /tmp/tempfile.txt
}
trap cleanup EXIT
touch /tmp/tempfile.txt
echo "Doing some work..."
sleep 5
```

REAL USE

Ensures temporary files or locks are always removed, even if a script is stopped early or crashes - preventing leftover clutter on servers.

7. Here-Documents (heredoc)

A heredoc lets you write multiple lines of text (like a config file or email body) directly inside a script, without many echo lines.



```
#!/bin/bash
cat <<EOF > config.txt
server_name=myapp
port=8080
env=production
EOF
echo "Config file created."
```

REAL USE

Generating configuration files (nginx configs, Docker Compose files, environment files) directly from a script during setup.

8. Debugging with set -x

set -x prints every command as it runs, with the actual values substituted in – extremely useful for finding out exactly where a script is going wrong.



```
#!/bin/bash
set -x # turn debugging ON
name="Rohan"
echo "Hello $name"
set +x # turn debugging OFF
```

REAL USE

When a script fails in production and the reason is not obvious, running it with set -x (or bash -x script.sh) shows exactly which line and value caused the failure.

9. jq - Working with JSON

Modern DevOps tools (AWS CLI, Docker, Kubernetes, APIs) return data in JSON format. jq is the standard command-line tool to read and filter JSON.



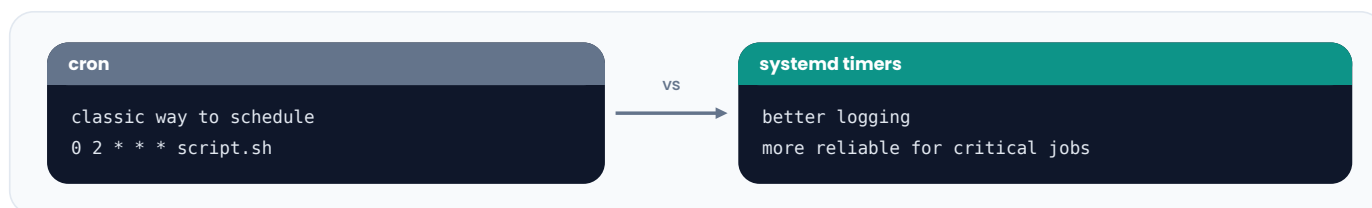
```
# Example: extract instance ID from AWS CLI JSON output
aws ec2 describe-instances | jq '.Reservations[0].Instances[0].InstanceId'
echo '{"name": "Rohan", "age": 25}' | jq '.name'
```

REAL USE

Almost every cloud automation script (like the AWS EC2 script in this repo) uses jq to pull out specific values from JSON API responses.

10. Scheduling Beyond Cron: systemd timers (mention)

cron is the classic way to schedule scripts, but modern Linux servers also use systemd timers, which offer better logging and reliability for critical jobs.



```
# View active timers on a modern Linux server
systemctl list-timers
```

REAL USE

Enterprise servers often prefer systemd timers over cron for critical scheduled jobs, since failures are logged in systemd's journal automatically.

11

The Grand Script

All lessons combined + line-by-line explanation



This final script is built specially for this guide. It logically combines a concept from every single lesson (1 to 19) into one working DevOps setup script, so you can see how all the small pieces fit together in a real-world automation.

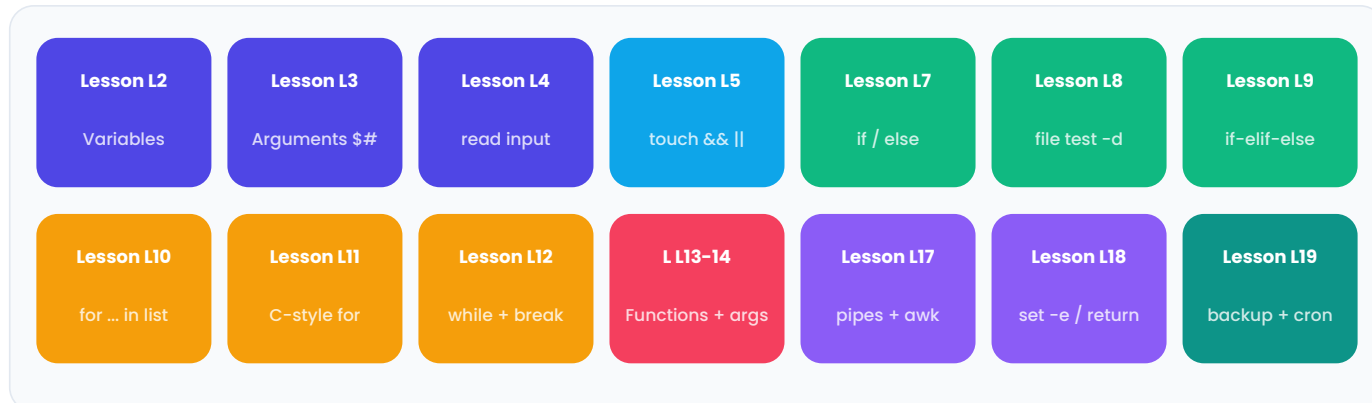


Fig 5. Which lesson powers which part of the Grand Script.

The Grand Script: every lesson, one file

Part 1: header and variables

```
#!/bin/bash
# =====
# grand_devops_script.sh
# A single script that uses every concept from Lessons 1-19
# Author: Sharad Verma | Based on Shell-Scripting-For-DevOps
# =====
set -e # Lesson 18: stop on any error

# ----- Lesson 2: Variables -----
APP_NAME="MyDevOpsApp"
BACKUP_DIR="$HOME/shell-backups"
LOG_FILE="$HOME/app_setup.log"
today=$(date +%d-%m-%Y) # command substitution
```

Part 2: functions

```
# ----- Lesson 13/14: Functions with arguments -----
log_message() {
  # $1 = message text
  echo "[${today}] $1" | tee -a "$LOG_FILE"
}

check_folder() {
  # Lesson 8: file test, Lesson 18: return on failure
  if [[ ! -d "$1" ]]
  then
    log_message "Folder $1 not found. Creating it..."
    mkdir -p "$1"
  fi
  return 0
}

create_backup() {
  # Lesson 19: capstone backup logic
  timestamp=$(date +%Y%m%d_%H%M%S)
  filename="backup_${timestamp}.tar.gz"
  tar -czf "$BACKUP_DIR/$filename" "$1"
  log_message "Backup created: $filename"
}

show_system_info() {
  # Lesson 17: pipes + awk
  log_message "Memory used: $(free -h | awk 'NR==2 {print $3}')"
  log_message "Disk used: $(df -h / | awk 'NR==2 {print $5}')"
}
```

Part 3: arguments, input and decisions

```
# ----- Lesson 3: Script arguments -----
if [[ $# -eq 0 ]] # Lesson 8: argument count check
then
    echo "Usage: $0 <folder-to-backup>"
    exit 1
fi
target_folder=$1

# ----- Lesson 4: User input -----
read -p "Enter your name to start setup: " user_name

# ----- Lesson 7: If-else -----
if [[ -z "$user_name" ]]
then
    echo "Name cannot be empty!"
    exit 1
else
    log_message "Setup started by $user_name for $APP_NAME"
fi

# ----- Lesson 9: if-elif-else (multi-branch) -----
read -p "Choose environment (dev/staging/prod): " env
if [[ "$env" == "dev" ]]
then
    log_message "Running in DEVELOPMENT mode"
elif [[ "$env" == "staging" ]]
then
    log_message "Running in STAGING mode"
elif [[ "$env" == "prod" ]]
then
    log_message "Running in PRODUCTION mode - extra caution enabled"
else
    log_message "Unknown environment, defaulting to dev"
fi
```

Part 4: loops and retry logic

```
# ----- Lesson 10: for loop (over a list) -----
required_tools=(git curl tar awk)
for tool in "${required_tools[@]}"
do
    if command -v "$tool" &>/dev/null
    then
        log_message "$tool is installed"
    else
        log_message "WARNING: $tool is missing"
    fi
done

# ----- Lesson 11: C-style for loop (counter) -----
for (( i=1; i<=3; i++ ))
do
    log_message "Pre-flight check $i of 3..."
    sleep 1
done
```

Part 4b: while loop with retry logic

```
# ----- Lesson 12: while loop (retry logic) -----
attempt=1
max_attempts=3
success=false
while [[ $attempt -le $max_attempts ]]
do
    log_message "Deployment attempt $attempt of $max_attempts..."
    # Simulated check - in real life this could be curl to a health endpoint
    if [[ -d "$target_folder" ]]
    then
        success=true
        break # Lesson 12: break keyword
    fi
    attempt=$((attempt + 1))
    sleep 1
done

if [[ "$success" == false ]]
then
    log_message "ERROR: Target folder never became ready. Exiting."
    exit 1
fi
```

Part 5: file operations and the real work

```
# ----- Lesson 16 style check + Lesson 5/6: file ops -----
check_folder "$BACKUP_DIR"
touch "$BACKUP_DIR/.setup_complete" && \
    log_message "Setup marker file created" || \
    log_message "Could not create marker file"

# ----- Run the real work -----
show_system_info
create_backup "$target_folder"

log_message "All done! $APP_NAME setup finished for $user_name in $env mode."
echo "Tip: schedule this with cron using: crontab -e"
# 0 2 * * * /bin/bash $HOME/grand_devops_script.sh /path/to/app >> /tmp/setup.log 2>&1
```

Line-by-line explanation of the Grand Script

Script part	Explanation
<code>set -e (top of file)</code>	Lesson 18. If any single command in this script fails, the whole script stops immediately instead of continuing with a broken state.
<code>APP_NAME, BACKUP_DIR, LOG_FILE, today</code>	Lesson 2. Variables store reusable values, including a timestamp captured using command substitution <code>\$(date +%d-%m-%Y)</code> .
<code>log_message() function</code>	Lesson 13 & 14. A reusable function that takes one argument (<code>\$1</code>) and writes a timestamped line to both the screen and a log file, using <code>tee -a</code> .
<code>check_folder() function</code>	Lesson 8, 13 & 18. Uses the <code>-d</code> file test to check if a folder exists, creates it if missing, and returns 0 to signal success back to the caller.
<code>create_backup() function</code>	Lesson 19. The capstone backup logic, reused here as a function – creates a timestamped tar.gz archive of the given folder.
<code>show_system_info() function</code>	Lesson 17. Uses pipes and awk to pull just the memory-used and disk-used values out of <code>free -h</code> and <code>df -h</code> .
<code>if [[\$# -eq 0]] ... exit 1</code>	Lesson 3 & 8. Validates that the script was given a folder argument; if not, it shows usage instructions and stops safely.
<code>read -p "Enter your name..."</code>	Lesson 4. Asks the user for their name while the script is running.

Script part	Explanation
<code>if [[-z "\$user_name"]]</code>	Lesson 7. Checks if the input was left empty (<code>-z</code> means 'string is empty') and stops the script if so.
<code>if / elif / elif / else</code> (environment)	Lesson 9. A multi-branch decision to configure the script differently for dev, staging or prod environments.
<code>for tool in "\${required_tools[@]}"</code>	Lesson 10 + Arrays (Additional Topics). Loops over a list of required tools and checks if each one is installed using <code>command -v</code> .
<code>for ((i=1; i<=3; i++))</code>	Lesson 11. Runs exactly 3 'pre-flight check' rounds using a counter-based loop, pausing 1 second between each.
<code>while [[\$attempt -le \$max_attempts]]</code>	Lesson 12. Retries up to 3 times, using <code>break</code> to exit early the moment the check succeeds - classic retry/polling logic.
<code>touch ... &&</code>	Lesson 5. Creates a marker file, logging success with <code>&&</code> or a warning with <code> </code> depending on whether touch worked.
Final <code>log_message</code> + cron tip	Lesson 19. Prints a final completion message and includes a ready-to-use cron line as a comment, so this script can be scheduled to run automatically.

12

Quick Revision Cheat Sheet

Everything on one page, fast



Variables & Substitution

<code>x="value"</code>	Assign a variable (no spaces around =)
<code>\$x</code> or <code>"\$x"</code>	Read a variable's value
<code>\$(command)</code>	Command substitution - capture output
<code>\$(expr)</code>	Arithmetic calculation

String Comparison

<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>-z "\$x"</code>	String is empty
<code>-n "\$x"</code>	String is not empty

File Tests

<code>-f file</code>	File exists
<code>-d dir</code>	Directory exists
<code>-e path</code>	File or directory exists
<code>-s file</code>	File exists and is not empty
<code>-x file</code>	File is executable

Functions

<code>name() { ... }</code>	Define a function
<code>name arg1 arg2</code>	Call a function with arguments
<code>return N</code>	Exit a function with a status code
<code>source file.sh</code>	Load functions/variables from another file

Operators & Redirection

<code>cmd1 && cmd2</code>	Run cmd2 only if cmd1 succeeds
<code>cmd1 cmd2</code>	Run cmd2 only if cmd1 fails
<code>cmd1 cmd2</code>	Pipe output of cmd1 into cmd2
<code>&>/dev/null</code>	Hide all command output

Script & Function Arguments

<code>\$0</code>	Script name
<code>\$1 \$2 ...</code>	1st, 2nd argument (script or function)
<code>\$#</code>	Number of arguments
<code>@</code>	All arguments
<code>\$?</code>	Exit code of the last command

Numeric Comparison

<code>-eq</code>	Equal to
<code>-ne</code>	Not equal to
<code>-gt / -ge</code>	Greater than / or equal
<code>-lt / -le</code>	Less than / or equal

Loops

<code>for x in list; do ... done</code>	Loop over a fixed list
<code>for ((i=1;i<=N;i++)); do ... done</code>	Loop N times with a counter
<code>while [[cond]]; do ... done</code>	Loop while condition is true
<code>break / continue</code>	Exit loop early / skip to next round

Error Handling

<code>exit 0 / exit 1</code>	Stop script: success / failure
<code>set -e</code>	Auto-stop script on any command failure
<code>if ! command; then</code>	Manually check if a command failed

Useful Commands

<code>chmod +x file.sh</code>	Make a script executable
<code>touch file</code>	Create an empty file
<code>tar -czf out.tar.gz dir/</code>	Compress a folder into an archive
<code>df -h / free -h</code>	Disk usage / memory usage
<code>awk '{print \$N}'</code>	Print column N of text output
<code>crontab -e / -l</code>	Edit / list scheduled cron jobs

THAT'S A WRAP!

Practice each lesson's task, rebuild the Grand Script yourself from memory, and you'll be comfortable with shell scripting for real DevOps work.

Original repo: <https://github.com/LondheShubham153/Shell-Scripting-For-DevOps>

Guide by: Sharad Verma